DDL used (exact SQL)

1) Create database — 01_create_database.sql

```
CREATE DATABASE IF NOT EXISTS cmaps;
USE cmaps;
```

2) Core tables — 02_create_tables.sql, 02b_create_test_tables.sql

```
USE cmaps;

CREATE TABLE IF NOT EXISTS train_cycles (
  dataset_id TINYINT NOT NULL,
  unit       INT     NOT NULL,
  cycle      INT     NOT NULL,
  setting_1 DOUBLE, setting_2 DOUBLE, setting_3 DOUBLE,
  sensor_1 DOUBLE, sensor_2 DOUBLE, sensor_3 DOUBLE, sensor_4 DOUBLE, sensor_5 DOUBLE,
  sensor_6 DOUBLE, sensor_7 DOUBLE, sensor_8 DOUBLE, sensor_9 DOUBLE, sensor_10 DOUBLE,
  sensor_11 DOUBLE, sensor_12 DOUBLE, sensor_13 DOUBLE, sensor_14 DOUBLE, sensor_15 DOUBLE,
  sensor_16 DOUBLE, sensor_17 DOUBLE, sensor_18 DOUBLE, sensor_19 DOUBLE, sensor_20 DOUBLE,
  sensor_21 DOUBLE,
  PRIMARY KEY (dataset_id, unit, cycle)
);

CREATE TABLE IF NOT EXISTS test_cycles (
  dataset_id TINYINT NOT NULL,
  unit       INT     NOT NULL,
  cycle      INT     NOT NULL,
  setting_1 DOUBLE, setting_2 DOUBLE, setting_3 DOUBLE,
  sensor_1 DOUBLE, sensor_2 DOUBLE, sensor_3 DOUBLE, sensor_4 DOUBLE, sensor_5 DOUBLE,
  sensor_6 DOUBLE, sensor_7 DOUBLE, sensor_8 DOUBLE, sensor_9 DOUBLE, sensor_10 DOUBLE,
  sensor_11 DOUBLE, sensor_12 DOUBLE, sensor_13 DOUBLE, sensor_14 DOUBLE, sensor_15 DOUBLE,
  sensor_16 DOUBLE, sensor_17 DOUBLE, sensor_18 DOUBLE, sensor_19 DOUBLE, sensor_20 DOUBLE,
  sensor_21 DOUBLE,
  PRIMARY KEY (dataset_id, unit, cycle)
);

CREATE TABLE IF NOT EXISTS rul_truth (
  dataset_id TINYINT NOT NULL,
```

```
  unit       INT     NOT NULL,
  rul        INT     NOT NULL,
  PRIMARY KEY (dataset_id, unit)
);

CREATE TABLE IF NOT EXISTS live_engine_input (
  dataset_id TINYINT NOT NULL,
  unit       INT     NOT NULL,
  cycle      INT     NOT NULL,
  sensor_2   DOUBLE,
  sensor_4   DOUBLE,
  sensor_11  DOUBLE,
  PRIMARY KEY (dataset_id, unit, cycle)
);
```

3) Constraints & Indexes — 02c_constraints_indexes.sql

```
USE cmaps;

ALTER TABLE train_cycles
  ADD CONSTRAINT chk_train_dataset_id CHECK (dataset_id BETWEEN 1 AND 4),
  ADD CONSTRAINT chk_train_cycle      CHECK (cycle >= 1);

ALTER TABLE test_cycles
  ADD CONSTRAINT chk_test_dataset_id CHECK (dataset_id BETWEEN 1 AND 4),
  ADD CONSTRAINT chk_test_cycle      CHECK (cycle >= 1);

ALTER TABLE live_engine_input
  ADD CONSTRAINT chk_live_dataset_id CHECK (dataset_id BETWEEN 1 AND 4),
  ADD CONSTRAINT chk_live_cycle      CHECK (cycle >= 1);

CREATE INDEX idx_train_unit ON train_cycles(unit);
CREATE INDEX idx_test_unit  ON test_cycles(unit);
CREATE INDEX idx_live_unit  ON live_engine_input(unit);
```

4) Dimensions & Foreign Keys — 02d_dimensions_fk.sql

```
USE cmaps;

CREATE TABLE IF NOT EXISTS datasets (
```

```
  dataset_id TINYINT PRIMARY KEY
) ENGINE=InnoDB;

INSERT IGNORE INTO datasets(dataset_id) VALUES (1),(2),(3),(4);

CREATE TABLE IF NOT EXISTS units (
  dataset_id TINYINT NOT NULL,
  unit       INT     NOT NULL,
  PRIMARY KEY(dataset_id, unit),
  CONSTRAINT fk_units_dataset FOREIGN KEY (dataset_id) REFERENCES datasets(dataset_id)
) ENGINE=InnoDB;

INSERT IGNORE INTO units(dataset_id, unit)
SELECT dataset_id, unit FROM train_cycles GROUP BY dataset_id, unit
UNION SELECT dataset_id, unit FROM test_cycles GROUP BY dataset_id, unit
UNION SELECT dataset_id, unit FROM rul_truth    GROUP BY dataset_id, unit
UNION SELECT dataset_id, unit FROM live_engine_input GROUP BY dataset_id, unit;

ALTER TABLE train_cycles
  ADD CONSTRAINT fk_train_dataset FOREIGN KEY (dataset_id) REFERENCES datasets(dataset_id),
  ADD CONSTRAINT fk_train_unit    FOREIGN KEY (dataset_id, unit) REFERENCES units(dataset_id, unit)

ALTER TABLE test_cycles
  ADD CONSTRAINT fk_test_dataset FOREIGN KEY (dataset_id) REFERENCES datasets(dataset_id),
  ADD CONSTRAINT fk_test_unit    FOREIGN KEY (dataset_id, unit) REFERENCES units(dataset_id, unit);

ALTER TABLE rul_truth
  ADD CONSTRAINT fk_rul_dataset FOREIGN KEY (dataset_id) REFERENCES datasets(dataset_id),
  ADD CONSTRAINT fk_rul_unit    FOREIGN KEY (dataset_id, unit) REFERENCES units(dataset_id, unit);

ALTER TABLE live_engine_input
  ADD CONSTRAINT fk_live_dataset FOREIGN KEY (dataset_id) REFERENCES datasets(dataset_id),
  ADD CONSTRAINT fk_live_unit    FOREIGN KEY (dataset_id, unit) REFERENCES units(dataset_id, unit);
```

Analytical Views (health, calibration, prediction, evaluation)

Health normalization — 04c_views_health_normalized.sql

```
USE cmaps;

CREATE OR REPLACE VIEW v_sensor_minmax AS
```

```
SELECT dataset_id,
       MIN(sensor_2) AS min_s2, MAX(sensor_2) AS max_s2,
       MIN(sensor_4) AS min_s4, MAX(sensor_4) AS max_s4,
       MIN(sensor_11) AS min_s11, MAX(sensor_11) AS max_s11
FROM train_cycles
GROUP BY dataset_id;

CREATE OR REPLACE VIEW v_engine_health_norm AS
SELECT
  c.dataset_id, c.unit, c.cycle,
  LEAST(GREATEST((c.sensor_2  - s.min_s2)  / NULLIF(s.max_s2  - s.min_s2,  0), 0), 1) AS n_s2,
  LEAST(GREATEST((c.sensor_4  - s.min_s4)  / NULLIF(s.max_s4  - s.min_s4,  0), 0), 1) AS n_s4,
  LEAST(GREATEST((c.sensor_11 - s.min_s11) / NULLIF(s.max_s11 - s.min_s11, 0), 0), 1) AS n_s11,
  ROUND(LEAST(GREATEST(100 - (50*n_s2 + 30*n_s4 + 20*n_s11), 0), 100), 2) AS health_score
FROM train_cycles c
JOIN v_sensor_minmax s USING (dataset_id);
```

Calibration & evaluation — 09c_sql_calibration_norm.sql

```
USE cmaps;

CREATE OR REPLACE VIEW v_train_max_cycle AS
SELECT dataset_id, unit, MAX(cycle) AS max_cycle
FROM train_cycles
GROUP BY dataset_id, unit;

CREATE OR REPLACE VIEW v_train_true_rul AS
SELECT h.dataset_id, h.unit, h.cycle, h.health_score,
       (m.max_cycle - h.cycle) AS true_rul
FROM v_engine_health_norm h
JOIN v_train_max_cycle m
  ON m.dataset_id = h.dataset_id AND m.unit = h.unit;

CREATE OR REPLACE VIEW v_regression_sufficient_stats_norm AS
SELECT dataset_id,
       COUNT(*) AS n,
       SUM(health_score) AS sum_x,
       SUM(true_rul)     AS sum_y,
       SUM(health_score*health_score) AS sum_xx,
       SUM(health_score*true_rul)     AS sum_xy
FROM v_train_true_rul
```

```
GROUP BY dataset_id;

CREATE OR REPLACE VIEW v_regression_params_final_norm AS
SELECT dataset_id,
       (sum_y / n) - (
         (CASE WHEN (n*sum_xx - sum_x*sum_x)=0 THEN 0
               ELSE (n*sum_xy - sum_x*sum_y)/(n*sum_xx - sum_x*sum_x) END) * (sum_x / n)
       ) AS a,
       (CASE WHEN (n*sum_xx - sum_x*sum_x)=0 THEN 0
             ELSE (n*sum_xy - sum_x*sum_y)/(n*sum_xx - sum_x*sum_x) END) AS b
FROM v_regression_sufficient_stats_norm;

CREATE OR REPLACE VIEW v_test_health_norm AS
SELECT c.dataset_id, c.unit, c.cycle,
  LEAST(GREATEST((c.sensor_2  - s.min_s2)  / NULLIF(s.max_s2  - s.min_s2,  0), 0), 1) AS n_s2,
  LEAST(GREATEST((c.sensor_4  - s.min_s4)  / NULLIF(s.max_s4  - s.min_s4,  0), 0), 0), 1) AS n_s4,
  LEAST(GREATEST((c.sensor_11 - s.min_s11) / NULLIF(s.max_s11 - s.min_s11, 0), 0), 1) AS n_s11,
  ROUND(LEAST(GREATEST(100 - (50*n_s2 + 30*n_s4 + 20*n_s11), 0), 100), 2) AS health_score
FROM test_cycles c
JOIN v_sensor_minmax s USING (dataset_id);

CREATE OR REPLACE VIEW v_test_last_health_norm AS
SELECT th.dataset_id, th.unit,
       MAX(th.cycle) AS last_cycle,
       SUBSTRING_INDEX(GROUP_CONCAT(th.health_score ORDER BY th.cycle DESC), ',', 1) + 0.0 AS last_
FROM v_test_health_norm th
GROUP BY th.dataset_id, th.unit;

CREATE OR REPLACE VIEW v_test_prediction_calibrated_norm AS
SELECT l.dataset_id, l.unit, l.last_cycle, l.last_health,
       GREATEST(ROUND(p.a + p.b * l.last_health, 0), 0) AS predicted_rul_cal
FROM v_test_last_health_norm l
JOIN v_regression_params_final_norm p USING (dataset_id);

CREATE OR REPLACE VIEW v_eval_test_cal_norm AS
SELECT c.dataset_id, c.unit, c.last_cycle, c.last_health,
       c.predicted_rul_cal,
       r.rul AS true_rul,
       (c.predicted_rul_cal - r.rul) AS error
FROM v_test_prediction_calibrated_norm c
JOIN rul_truth r
  ON r.dataset_id = c.dataset_id AND r.unit = c.unit;
```

```
CREATE OR REPLACE VIEW v_eval_metrics_cal_norm AS
SELECT dataset_id,
       COUNT(*) AS n_units,
       ROUND(AVG(ABS(error)), 2) AS mae,
       ROUND(SQRT(AVG(POWER(error, 2))), 2) AS rmse
FROM v_eval_test_cal_norm
GROUP BY dataset_id;
```

Calibrated history status — 05e_predictions_calibrated_norm.sql

```
USE cmaps;

CREATE OR REPLACE VIEW v_engine_last_health_norm AS
SELECT dataset_id, unit,
       MAX(cycle) AS last_cycle,
       SUBSTRING_INDEX(GROUP_CONCAT(health_score ORDER BY cycle DESC), ',', 1) + 0.0 AS last_health
FROM v_engine_health_norm
GROUP BY dataset_id, unit;

CREATE OR REPLACE VIEW v_engine_prediction_calibrated_norm AS
SELECT l.dataset_id, l.unit, l.last_cycle, l.last_health,
       GREATEST(ROUND(p.a + p.b * l.last_health, 0), 0) AS estimated_cycles_left
FROM v_engine_last_health_norm l
JOIN v_regression_params_final_norm p USING (dataset_id);

CREATE OR REPLACE VIEW v_engine_status_calibrated_norm AS
SELECT dataset_id, unit, last_cycle, last_health AS avg_health,
       estimated_cycles_left,
       CASE WHEN estimated_cycles_left < 10 THEN 'FAIL SOON'
            WHEN estimated_cycles_left < 25 THEN 'WARNING'
            ELSE 'HEALTHY' END AS status
FROM v_engine_prediction_calibrated_norm;
```

DML used (exact SQL)

Bulk Load CMAPSS — 03a_load_cmapss_data.sql

```
USE cmaps;
SET GLOBAL local_infile = 1;

LOAD DATA LOCAL INFILE 'e:/DBMS-2/CMAPSSData/train_FD001.txt'
INTO TABLE train_cycles
FIELDS TERMINATED BY ' ' LINES TERMINATED BY '\n'
(@c1,@c2,@c3,@c4,@c5,@c6,@c7,@c8,@c9,@c10,@c11,@c12,@c13,@c14,@c15,@c16,@c17,@c18,@c19,@c20,@c21,@c
SET dataset_id=1,
    unit=@c1, cycle=@c2, setting_1=@c3, setting_2=@c4, setting_3=@c5,
    sensor_1=@c6, sensor_2=@c7, sensor_3=@c8, sensor_4=@c9, sensor_5=@c10,
    sensor_6=@c11, sensor_7=@c12, sensor_8=@c13, sensor_9=@c14, sensor_10=@c15,
    sensor_11=@c16, sensor_12=@c17, sensor_13=@c18, sensor_14=@c19, sensor_15=@c20,
    sensor_16=@c21, sensor_17=@c22, sensor_18=@c23, sensor_19=@c24, sensor_20=@c25,
    sensor_21=@c26;

SET GLOBAL local_infile = 0; -- secure after load
```

Insert / Update / Delete live reading

```
INSERT INTO live_engine_input VALUES (1, 999, 1, 78.0, 35.0, 20.0);

UPDATE live_engine_input
SET sensor_2 = 76.5
WHERE dataset_id=1 AND unit=999 AND cycle=1;

DELETE FROM live_engine_input
WHERE dataset_id=1 AND unit=999 AND cycle=1;
```

Validation (used in 10_verify.sql)

```
-- Health range and total rows
SELECT COUNT(*) AS row_count,
       ROUND(MIN(avg_health),2) AS min_h,
       ROUND(MAX(avg_health),2) AS max_h
FROM v_engine_status_calibrated_norm;

-- Test metrics
SELECT dataset_id, n_units, mae, rmse
```

```
FROM v_eval_metrics_cal_norm
ORDER BY dataset_id;
```

Observed: row_count=710; min_h≈0.24, max_h≈99.45; FD001 MAE≈38.60, RMSE≈49.69 (similar for FD002–FD004).

Generated from working project scripts in e:/DBMS-2/dbms_robot_project/.